POLYMORPHISM

Polymorphism means "many forms". It refers to the ability of an entity (like a function or object) to perform different actions based on the context.

Overloading

We can use same operator or methods for different purposes.

There are 3 types of Overloading

1) Operator Overloading

2) Method Overloading

3) Constructor Overloading

1) Operator Overloading:

- We can use the same operator for multiple purposes, which is nothing but operator overloading.
- Python supports operator overloading.

Example 1:

+ operator can be used for Arithmetic addition and String concatenation

print(10+20) → 30

print('python'+'java') → pythonjava

Example 2:

* operator can be used for multiplication and string repetition purposes.

print(10*20) → 200

print('python'*3) → pythonpythonpython

Demo program to use + operator for our class objects:

```
class Book:
    def __init__(self,pages):
        self.pages=pages

b1 = Book(100)
b2 = Book(50)

print(b1+b2)
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 8, in <module>
TypeError: unsupported operand type(s) for +: 'Book' and 'Book'

=== Code Exited With Errors ===
```

We can overload + operator to work with Book objects also. i.e Python supports Operator Overloading.

For every operator Magic Methods are available. To overload any operator we have to override that Method in our class.

Internally + operator is implemented by using __add__() method.This method is called magic method for + operator. We have to override this method in our class.

The following is the list of operators and corresponding magic methods.

1) + → object.__add__(self,other)
2) - → object.__sub__(self,other)
3) * → object.__mul__(self,other)
4) / → object.__div__(self,other)
5) // → object.__floordiv__(self,other)
6) % → object.__mod__(self,other)
7) ** → object.__pow__(self,other)
8) += → object.__iadd__(self,other)
9) -= → object.__isub__(self,other)
10) *= → object.__imul__(self,other)
11) /= → object.__idiv__(self,other)
12) //= → object.__ifloordiv__(self,other)

13) %= → object.__imod__(self,other)
14) **= → object.__ipow__(self,other)
15) < → object.__lt__(self,other)
16) <= → object.__le__(self,other)
17) > → object.__gt__(self,other)
18) >= → object.__ge__(self,other)
19) == → object.__eq__(self,other)
20) != → object.__ne__(self,other)

```
class Book:
    def __init__(self,pages):
        self.pages=pages

    def __add__(self,other):
        return self.pages+other.pages

b1 = Book(100)
b2 = Book(50)

print(b1+b2)  → 150
```

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    # Overload '<' (Less Than)
    def __lt__(self, other):
        return self.pages < other.pages

    # Overload '==' (Equal To)
    def __eq__(self, other):
        return self.pages == other.pages

b1 = Book("Python Basics", 100)
b2 = Book("Mastering AI", 300)

print(b1 < b2)   # Output: True (calls b1.__lt__(b2))
print(b1 == b2)  # Output: False (calls b1.__eq__(b2))
```

python

```python
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    # Overload '<' (Less Than)
    def __lt__(self, other):
        return self.pages < other.pages

    # Overload '==' (Equal To)
    def __eq__(self, other):
        return self.pages == other.pages

b1 = Book("Python Basics", 100)
b2 = Book("Mastering AI", 300)

print(b1 < b2)   # Output: True (calls b1.__lt__(b2))
print(b1 == b2)  # Output: False (calls b1.__eq__(b2))
```

2) Method Overloading:

- If 2 methods having same name but different type of arguments, then those methods are said to be overloaded methods.

- But in Python Method overloading is not possible.

- If we are trying to declare multiple methods with same name and different number of arguments, then Python will always consider only last method.

```
def m1(self):
    pass
```

```
def m1(self, a, b):
    return a+b
```

3) Constructor Overloading:

Constructor overloading is not possible in Python.
If we define multiple constructors, then the last constructor will be considered.

Example:

```
class Test:
    def __init__(self):
        print('No-Arg Constructor')

    def __init__(self,a):
        print('One-Arg constructor')

    def __init__(self,a,b):
        print('Two-Arg constructor')

#t1=Test()
#t1=Test(10)
t1=Test(10,20)  → 'Two-Arg constructor'
```

Method Overriding

Whatever members available in the parent class are by default available to the child class through inheritance. If the child class not satisfied with parent class implementation, then child class is allowed to redefine that method in the child class based on its requirement. This concept is called overriding.

Overriding concept applicable for both methods and constructors.

python

```python
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    # Overriding the parent's speak() method
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    # Overriding the parent's speak() method
    def speak(self):
        print("Cat meows")

# Instances
a = Animal()
d = Dog()
c = Cat()

a.speak()  # Output: Animal makes a sound
d.speak()  # Output: Dog barks
c.speak()  # Output: Cat meows
```

Demo Program for Constructor Overriding:

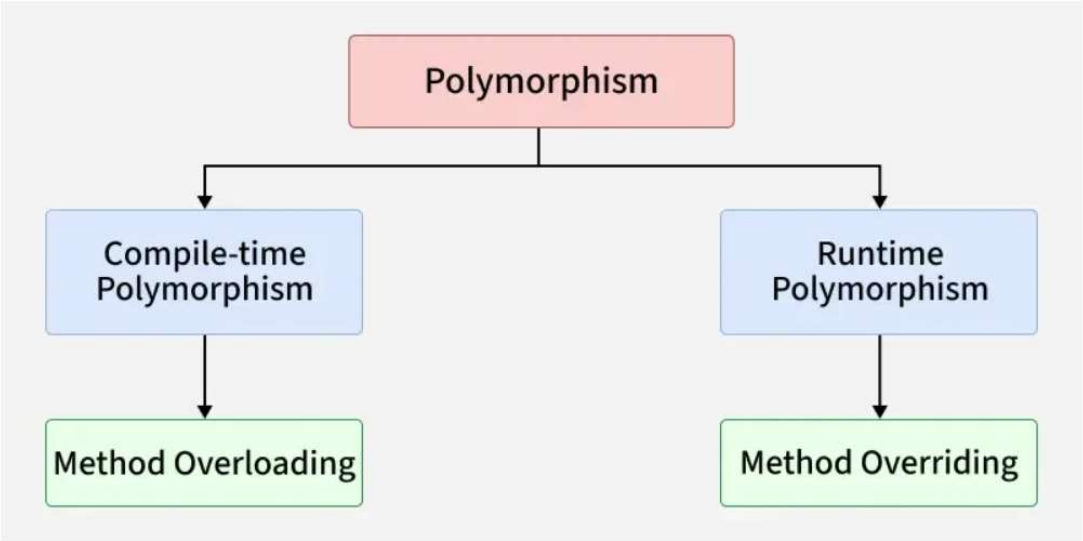
```
1) class P:
2)     def __init__(self):
3)         print('Parent Constructor')
4)
5) class C(P):
6)     def __init__(self):
7)         print('Child Constructor')
8)
9) c=C()
```

Output: **Child Constructor**
In the above example,if child class does not contain constructor then parent class constructor will be executed

From child class constuctor we can call parent class constructor by using super() method.

Types of Polymorphism

Polymorphism in Python refers to ability of the same method or operation to behave differently based on object or context. It mainly includes compile-time and runtime polymorphism.



Polymorphism

2. Runtime Polymorphism (Overriding)

Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it.

In Python, this happens through **Method Overriding** a child class provides its own version of a method already defined in the parent class. Since Python is dynamic, it supports this, allowing same method call to behave differently for different object types.

Data Abstraction

Abstraction hides the internal implementation details while exposing only the necessary functionality.

Abstract Method:

- Sometimes we don't know about implementation, still we can declare a method. Such types of methods are called abstract methods. i.e abstract method has only declaration but not implementation.
- In python we can declare abstract method by using @abstractmethod decorator as follows.
- @abstractmethod

```
def m1(self):
    pass
```

- @abstractmethod decorator present in abc module. Hence compulsory we should import abc module,otherwise we will get error.

- abc → abstract base class module

Abstract class:

Sometimes implementation of a class is not complete, such type of partially implementation classes are called abstract classes. Every abstract class in Python should be derived from ABC class which is present in abc module.

```
class Shape(ABC):
    @abstractmethod
    def area(self):
        """This method must be implemented by all subclasses."""
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side * self.side

# s = Shape()    # This would raise a TypeError
sq = Square(5)
print(sq.area()) # Output: 25
```

Note: If we are extending abstract class and does not override its abstract method then child class is also abstract and instantiation is not possible.

```
1) from abc import *
2) class Vehicle(ABC):
3)     @abstractmethod
4)     def noofwheels(self):
5)         pass
6)
7) class Bus(Vehicle):
8)     def noofwheels(self):
9)         return 7
10)
11) class Auto(Vehicle):
12)     def noofwheels(self):
13)         return 3
14) b=Bus()
15) print(b.noofwheels())#7
16)
17) a=Auto()
18) print(a.noofwheels())#3
```

Note: Abstract class can contain both abstract and non-abstract methods also.

Ques:-- Is this possible ?

```
from abc import *
class Vehicle(ABC):
   @abstractmethod
   def noofwheels(self):
         pass
class Bus(Vehicle):
        pass
b=Bus()
```

main.py

```python
from abc import *
class Vehicle(ABC):

    @abstractmethod
    def noofwheels(self):
        pass

class Bus(Vehicle):
    pass

b=Bus()
```

Output

```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 9, in <module>
TypeError: Can't instantiate abstract class Bus without an implementation for abstract
    method 'noofwheels'

=== Code Exited With Errors ===
```

If we are extending abstract class and does not override its abstract method then child class is also abstract and instantiation is not possible.

Interfaces In Python:

In general if an abstract class contains only abstract methods such type of abstract class is considered as interface.

test.py

```
1) from abc import *
2) class Printer(ABC):
3)     @abstractmethod
4)     def printit(self,text):pass
5)
6)     @abstractmethod
7)     def disconnect(self):pass
8)
9) class EPSON(Printer):
10)    def printit(self,text):
11)        print('Printing from EPSON Printer...')
12)        print(text)
13)    def disconnect(self):
14)        print('Printing completed on EPSON Printer...')
15)
16) class HP(Printer):
17)    def printit(self,text):
18)        print('Printing from HP Printer...')
19)        print(text)
20)    def disconnect(self):
21)        print('Printing completed on HP Printer...')
22)
23) with open('config.txt','r') as f:
24)    pname=f.readline()
25)
26) classname=globals()[pname]
27) x=classname()
28) x.printit('This data has to print...')
29) x.disconnect()
```

Output:

```
Printing from EPSON Printer...
This data has to print...
Printing completed on EPSON Printer...
```

In Python, interfaces are typically implemented using Abstract Base Classes (ABCs) from the built-in `abc` module. This approach provides a formal contract that forces subclasses to implement specific methods, promoting code consistency and maintainability.

```python
from abc import ABC, abstractmethod

# Define the interface using an Abstract Base Class (ABC)
class Vehicle(ABC):
    """
    An interface (abstract base class) for vehicles.
    """
    @abstractmethod
    def start_engine(self):
        """
        Abstract method to start the vehicle's engine.
        Must be implemented by all concrete subclasses.
        """
        pass

    @abstractmethod
    def stop_engine(self):
        """
        Abstract method to stop the vehicle's engine.
        Must be implemented by all concrete subclasses.
        """
        pass
```

```python
# Implement the interface in a concrete class
class Car(Vehicle):
    """
    A concrete implementation of the Vehicle interface.
    """
    def start_engine(self):
        print("Car engine started with a key turn.")

    def stop_engine(self):
        print("Car engine stopped.")

# A function that operates on any object that adheres to the Vehicle interface
def operate_vehicle(vehicle: Vehicle):
    vehicle.start_engine()
    vehicle.stop_engine()

# Create an instance of the concrete class
my_car = Car()

# Use the function with the car object
operate_vehicle(my_car)

# Output:
# Car engine started with a key turn.
# Car engine stopped.
```

What is the __str__ method in Python?

The `__str__` **method** in Python represents the class objects as a string – it can be used for classes. The `__str__` method should be defined in a way that is easy to read and outputs all the members of the class. This method is also used as a debugging tool when the members of a class need to be checked.

The `__str__` method is called when the following functions are invoked on the object and return a string:

- The `print()` method
- The `str()` method

If we have not defined the `__str__`, then it will call the `__repr__` method. The `__repr__` **method** returns a string that describes the pointer of the object by default (if the programmer does not define it).

```
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def __str__(self):
        """
        Returns the user-friendly string representation of the Book object.
        """
        return f"'{self.title}' by {self.author}, {self.year}"

# Create an instance of the Book class
book = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams", 1979)

# The __str__ method is automatically called when using print()
print(book)

# The __str__ method is also called when using the str() function
book_string = str(book)
print(book_string)
```

main.py

```python
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year


# Create an instance of the Book class
book = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams", 1979)

# The __str__ method is automatically called when using print()
print(book)

# The __str__ method is also called when using the str() function
book_string = str(book)
print(book_string)

```

Output

```
<__main__.Book object at 0x7ce01b4c1be0>
<__main__.Book object at 0x7ce01b4c1be0>

=== Code Execution Successful ===
```

main.py

```python
class Book:
    def __init__(self, title, author, year):
        self.title = title
        self.author = author
        self.year = year

    def __str__(self):
        """
        Returns the user-friendly string representation of the Book object.
        """
        return f"'{self.title}' by {self.author}, {self.year}"

# Create an instance of the Book class
book = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams", 1979)

# The __str__ method is automatically called when using print()
print(book)

# The __str__ method is also called when using the str() function
book_string = str(book)
print(book_string)

```

Output

```
'The Hitchhiker's Guide to the Galaxy' by Douglas Adams, 1979
'The Hitchhiker's Guide to the Galaxy' by Douglas Adams, 1979

=== Code Execution Successful ===
```

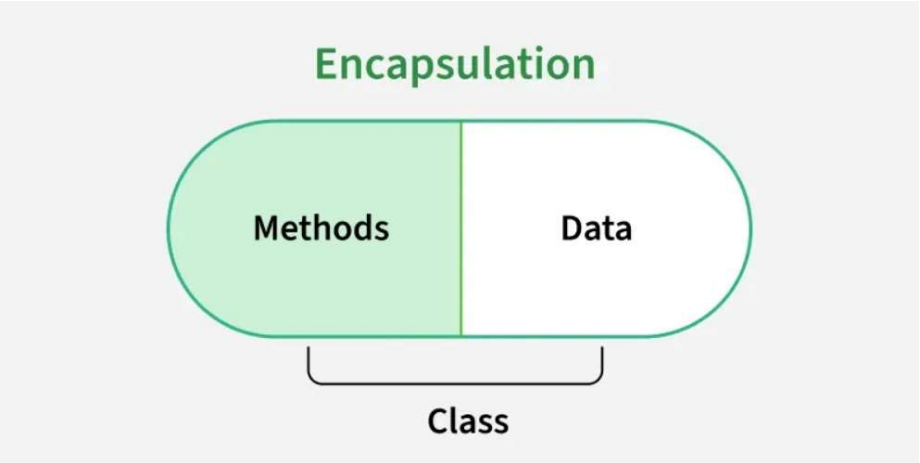
Python Encapsulation

Encapsulation is about protecting data inside a class.

It means keeping data (properties) and methods together in a class, while controlling how the data can be accessed from outside the class.

This prevents accidental changes to your data and hides the internal details of how your class works.

- The idea of encapsulation is to bind the data members and methods into a single unit.

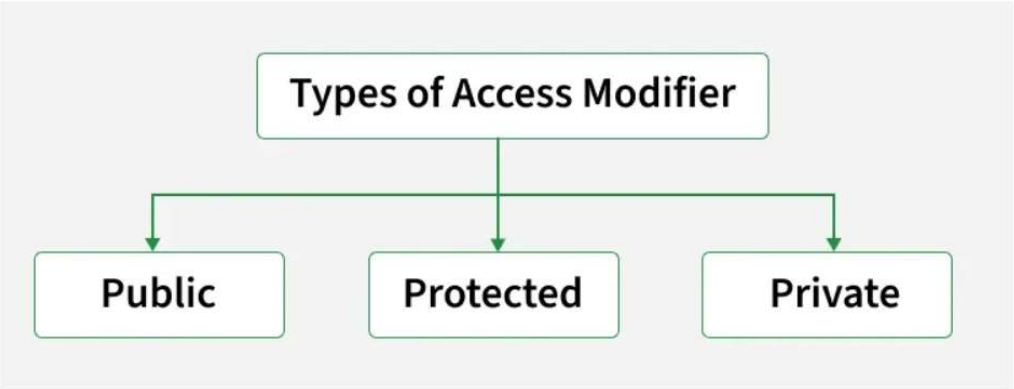


Why do we need Encapsulation?

- Protects data from unauthorized access and accidental modification.
- Controls data updates using getter/setter methods with validation.
- Enhances modularity by hiding internal implementation details.

Access Specifiers

Access specifiers define how class members (variables and methods) can be accessed from outside the class. They help in implementing encapsulation by controlling the visibility of data. There are three types of access specifiers:



Types of Access Modifiers

1. Public Members

Public members are variables or methods that can be accessed from anywhere inside the class, outside the class or from other modules. By default, all members in Python are public. They are defined without any underscore prefix (e.g., self.name).

Example: This example shows how a public attribute (name) and a public method (display_name) can be accessed from outside the class using an object.

```
class Employee:
    def __init__(self, name):
        self.name = name   # public attribute

    def display_name(self):   # public method
        print(self.name)

emp = Employee("John")
emp.display_name()   # Accessible
print(emp.name)      # Accessible
```

main.py

```python
class Employee:
    def __init__(self, name):
        self.name = name   # public attribute

    def display_name(self):   # public method
        print(self.name)

emp = Employee("John")
emp.display_name()   # Accessible
print(emp.name)      # Accessible
```

Output

```
John
John

=== Code Execution Successful ===
```

2. Protected members

Protected members are variables or methods that are intended to be accessed only within the class and its subclasses. They are not strictly private but should be treated as internal. In Python, protected members are defined with a single underscore prefix (e.g., self._name).

Example: This example shows how a protected attribute **(_age)** can be accessed within a subclass, demonstrating that protected members are meant for use within the class and its subclasses.

```python
class Employee:
    def __init__(self, name, age):
        self.name = name        # public
        self._age = age         # protected

class SubEmployee(Employee):
    def show_age(self):
        print("Age:", self._age)   # Accessible in subclass

emp = SubEmployee("Ross", 30)
print(emp.name)         # Public accessible
emp.show_age()          # Protected accessed through subclass
```

main.py

```python
class Employee:
    def __init__(self, name, age):
        self.name = name        # public
        self._age = age         # protected

class SubEmployee(Employee):
    def show_age(self):
        print("Age:", self._age)   # Accessible in subclass

emp = SubEmployee("Ross", 30)
print(emp.name)        # Public accessible
emp.show_age()         # Protected accessed through subclass
```

Output

```
Ross
Age: 30

=== Code Execution Successful ===
```

3. Private members

Private members are variables or methods that cannot be accessed directly from outside the class. They are used to restrict access and protect internal data. In Python, private members are defined with a double underscore prefix (e.g., self.__salary). Python applies name mangling by internally renaming them (e.g., __salary becomes _ClassName__salary) to prevent direct access.

Example: This example shows how a private attribute **(__salary)** is accessed within the class using a public method, demonstrating that private members cannot be accessed directly from outside the class.

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name          # public
        self.__salary = salary    # private

    def show_salary(self):
        print("Salary:", self.__salary)

emp = Employee("Robert", 60000)
print(emp.name)          # Public accessible
emp.show_salary()        # Accessing private correctly
# print(emp.__salary)    # Error: Not accessible directly
```

```python
class Employee:
    def __init__(self, name, salary):
        self.name = name          # public
        self.__salary = salary    # private

    def show_salary(self):
        print("Salary:", self.__salary)

emp = Employee("Robert", 60000)
print(emp.name)          # Public accessible
emp.show_salary()        # Accessing private correctly
print(emp.__salary)    # Error: Not accessible directly
```

```
Robert
Salary: 60000
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 12, in <module>
AttributeError: 'Employee' object has no attribute '__salary'

=== Code Exited With Errors ===
```

Example:

```python
class Test:
    x=10
    _y=20
    __z=30
    def m1(self):
        print(Test.x)
        print(Test._y)
        print(Test.__z)

t=Test()
t.m1()
print(Test.x)
print(Test._y)
print(Test.__z)
```

```python
class Test:
    x=10
    _y=20
    __z=30
    def m1(self):
        print(Test.x)
        print(Test._y)
        print(Test.__z)

t=Test()
t.m1()
print(Test.x)
print(Test._y)
print(Test.__z)
```

```
10
20
30
10
20
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 14, in <module>
AttributeError: type object 'Test' has no attribute '__z'

=== Code Exited With Errors ===
```

How to Access Private Variables from Outside of the Class:

We cannot access private variables directly from outside of the class.
But we can access indirectly as follows `objectreference._classname__variablename`

main.py

```
class Test:
    x=10
    _y=20
    __z=30
    def m1(self):
        print(Test.x)
        print(Test._y)
        print(Test.__z)

t=Test()
#t.m1()
#print(Test.x)
#print(Test._y)
print(t._Test__z)
#print(Test.__z)
```

Share Run

Output

```
30

=== Code Execution Successful ===
```